



Plataforma de mensagens baseada em tópicos

Relatório de Sistemas Operativos
No âmbito da Licenciatura em Engenharia Informática (Europeu)

Trabalho Prático - Programa em C para UNIX
António Correia a2022141330

15 de dezembro de 2024

Índice

1. INTRODUÇÃO	4
2. MAKEFILE	5
3. UTIL.H	5
4. FEED.C.....	5
4.1. Função main.....	6
4.2. Função recebe_resposta (thread)	8
4.3. Função escreve_mensagem_manager	8
4.4. Função handle_exit	9
5. MANAGER.C.....	9
5.1. Declarações iniciais.....	9
5.2. Função main.....	10
5.3. Ciclo principal	10
5.4. Função add_user.....	12
5.5. Função encontra_nome	13
5.6. Função mostra_users	13
5.7. Função cria_topico	13
5.8. Função procura_topico.....	13
5.9. Função procura_utilizador_topicos	13
5.10. Função apaga_utilizador_topicos	13
5.11. Função apaga_topico	14
5.12. Função mostra_topicos	14
5.13. Função lock_topico / unlock_topico.....	14
5.14. Função topico_islocked	14
5.15. Função add_mensagem_persistente	14
5.16. Função mostra_mensagens_persistentes	14
5.17. Função utilizador_cancela_sub	15
5.18. Função utilizador_subscreve_topico	15
5.19. Função envia_mensagem_todos	15
5.20. Função envia_mensagem.....	16
5.21. Função salva_mensagens_persistentes.....	17
5.22. Função carrega_mensagens_persistentes.....	17
5.23. Função handle_exit	18
5.24. Função envia_mensagem_topico	18
5.25. Função envia_mensagens_persistentes	19

5.26. Threads	19
5.26.1. decrementa_tempo_mensagens	19
5.26.2. comandos_manager	20
6. CONCLUSÃO	21

1. INTRODUÇÃO

Este relatório apresenta o desenvolvimento de uma plataforma de mensagens baseada em tópicos no âmbito da unidade curricular de Sistemas Operativos. O objetivo principal foi aplicar conceitos de programação em C no ambiente UNIX/Linux, utilizando mecanismos como named pipes, manipulação de sinais, threads, etc...

A plataforma é composta por dois programas principais:

- Manager: responsável por gerir tópicos, mensagens e utilizadores.
- Feed: interface de utilizador que permite enviar e receber mensagens, bem como subscrever e cancelar subscrições em tópicos.

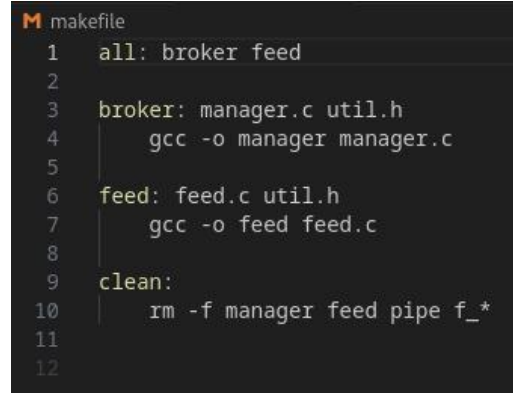
Durante a execução do trabalho, foi dada ênfase ao uso correto das chamadas ao sistema, à implementação de uma comunicação eficiente entre processos e à gestão de recursos do sistema, garantindo robustez e conformidade com os requisitos especificados pelo docente. Este relatório descreve a arquitetura geral, as decisões de implementação e as funcionalidades desenvolvidas, bem como os desafios enfrentados ao longo do projeto.

2. MAKEFILE

O ficheiro Makefile é responsável por automatizar o processo de compilação e limpeza dos executáveis no projeto.

Targets definidos:

- **all:** alvo principal que compila ambos os programas (broker e feed);
- **broker:** compila o programa do servidor (Manager) a partir de `manager.c`;
- **feed:** compila o programa do cliente (Feed) a partir de `feed.c`;
- **clean:** remove os binários gerados (manager, feed) e quaisquer FIFOs criados (pipe, `f_*`).



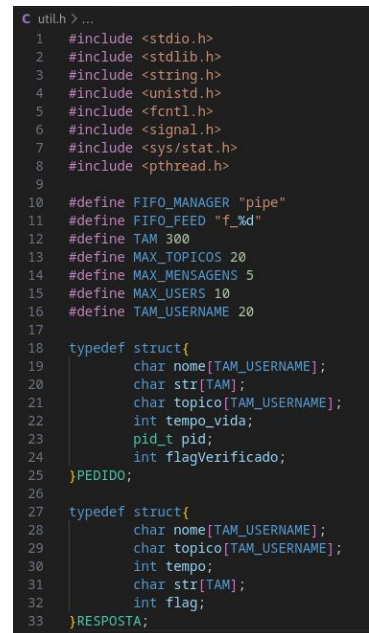
```
M makefile
1  all: broker feed
2
3  broker: manager.c util.h
4      gcc -o manager manager.c
5
6  feed: feed.c util.h
7      gcc -o feed feed.c
8
9  clean:
10     rm -f manager feed pipe f_*
11
12
```

Figura 1: Targets definidos no ficheiro Makefile.

3. UTIL.H

O arquivo `util.h` funciona como um cabeçalho para o programa, contendo definições de constantes, estruturas de dados e includes necessários para a implementação do cliente (**Feed**) e do servidor (**Manager**).

- **Constantes e Definições:**
 - Define nomes e formatos dos FIFOs utilizados na comunicação (FIFO_MANAGER e FIFO_FEED).
 - Especifica limites importantes, como tamanho máximo de mensagens (TAM), nome de utilizadores/tópicos (TAM_USERNAME), e quantidades máximas de tópicos, mensagens e utilizadores.
- **Estruturas de Dados:**
 - **PEDIDO:** Estrutura usada pelo cliente para enviar solicitações ao servidor, contendo informações como nome do utilizador, mensagem, tópico, tempo de vida, e PID.
 - **RESPOSTA:** Estrutura usada pelo servidor para enviar respostas aos clientes, incluindo informações da mensagem.
- **Includes Necessários:** adiciona as bibliotecas padrão necessárias para manipulação de arquivos, strings, sinais, threads, etc.



```
C util.h > ...
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <fcntl.h>
6  #include <signal.h>
7  #include <sys/stat.h>
8  #include <pthread.h>
9
10 #define FIFO_MANAGER "pipe"
11 #define FIFO_FEED "f_%d"
12 #define TAM 300
13 #define MAX_TOPICOS 20
14 #define MAX_MENSAGENS 5
15 #define MAX_USERS 10
16 #define TAM_USERNAME 20
17
18 typedef struct{
19     char nome[TAM_USERNAME];
20     char str[TAM];
21     char topico[TAM_USERNAME];
22     int tempo_vida;
23     pid_t pid;
24     int flagVerificado;
25 }PEDIDO;
26
27 typedef struct{
28     char nome[TAM_USERNAME];
29     char topico[TAM_USERNAME];
30     int tempo;
31     char str[TAM];
32     int flag;
33 }RESPOSTA;
```

Figura 2: Declarações do arquivo `util.h`.

4. FEED.C

O **Feed** implementa um cliente funcional e robusto para o sistema, atendendo aos requisitos do enunciado.

Na seguinte imagem estão representadas as declarações de variáveis globais.

```

1  #include "util.h"
2
3  int fd = -1;           // Descritor do FIFO do servidor
4  int fd_feed = -1;      // Descritor do FIFO do cliente
5  char fifo_feed[20];    // Nome do FIFO do cliente
6  int fifo_created = 0;  // Flag para saber se o FIFO foi criado
7
8

```

Figura 3: Declarações de constantes e variáveis globais.

As primeiras linhas do feed.c definem os principais recursos do cliente, garantindo que ele possa interagir com o servidor. Foi assim feito, para facilitar o fechamento e eliminação de todos os FIFOs no final do programa.

4.1. Função main

Configuração Inicial:

- Verifica se o nome do utilizador foi fornecido como argumento no terminal;
- Valida a existência do FIFO do servidor para garantir que o Manager está ativo.

Preparação do FIFO do Cliente:

- Cria um FIFO exclusivo para o cliente, cujo nome é baseado no PID do processo (f_%d);
- Abre o FIFO em modo leitura/escrita para evitar bloqueios durante a leitura e escrita.

Conexão com o servidor:

- Abre o FIFO do servidor para enviar pedidos;
- Envia um pedido de login com o nome do utilizador e o PID do cliente;
- Aguarda a validação do login (resultado vem na flag da estrutura RESPOSTA), exibindo uma mensagem de sucesso ou erro.

Recebimento de mensagens

- Cria uma thread (recebe_resposta) para receber mensagens do servidor.

Loop principal:

- Para todos os comandos exceto “help” (explicado mais adiante), é enviado um PEDIDO para o manager. A leitura dos comandos é feita através de um fgets e posteriormente repartida usando sscanf. No caso do comando “msg”, se for

```

72 int main(int argc, char *argv[]) {
73     int res;
74     RESPOSTA r;
75     PEDIDO login;
76
77     // Registrar manipulador para SIGINT
78     signal(SIGINT, handle_exit);
79
80     if (argc <= 1) {
81         perror("ERRO) Precisa de especificar um nome!!\n");
82         exit(1);
83     }
84
85     if (access(FIFO_MANAGER, F_OK) != 0) {
86         perror("ERRO) O servidor não está a funcionar!!\n");
87         exit(1);
88     }
89
90     // Criar nome para o FIFO do cliente
91     sprintf(fifo_feed, FIFO_FEED, getpid());
92
93     // Criar FIFO do cliente
94     if (mkfifo(fifo_feed, 0600) == -1) {
95         perror("ERRO) Não foi possível criar o FIFO do cliente");
96         exit(2);
97     }
98     fifo_created = 1; // FIFO foi criado com sucesso
99
100    // Abrir FIFO do cliente
101    fd_feed = open(fifo_feed, O_RDWR);
102    if (fd_feed == -1) {
103        perror("ERRO) Não foi possível abrir o FIFO do cliente");
104        unlink(fifo_feed);
105        exit(3);
106    }
107
108    // Abrir FIFO do servidor
109    fd = open(FIFO_MANAGER, O_WRONLY);
110    if (fd == -1) {
111        perror("ERRO) Não foi possível abrir o FIFO do servidor");
112        close(fd_feed);
113        unlink(fifo_feed);
114        exit(4);
115    }
116

```

Figura 4: Função main: configuração inicial, preparação do FIFO do cliente e conexão com o servidor.

```

117 // Enviar login
118 strcpy(login.nome, argv[1]);
119 login.pid = getpid();
120 login.flagVerificado = 0;
121
122 res = write(fd, login, sizeof(PEDIDO));
123 if (res == sizeof(PEDIDO)) {
124     printf("TENTATIVA DE LOGIN ENVIADA COM SUCESSO\n");
125     res = read(fd_feed, &r, sizeof(RESPOSTA));
126     if (res > 0) {
127         if (r.flag == 0) {
128             printf("MENSAGEM RECEBIDA. 'Ns' - %s\n", r.str, r.nome);
129             printf("ERRO) Tentativa de login falhou\n");
130             close(fd);
131             close(fd_feed);
132             unlink(fifo_feed);
133             exit(3);
134         } else {
135             printf("MENSAGEM RECEBIDA. 'Ns' - %s\n", r.str, r.nome);
136             printf("LOGIN SUCESSO\n");
137             login.flagVerificado = 1;
138         }
139     } else {
140         printf("ERRO) Impossível ler resposta do login\n");
141     }
142 } else {
143     printf("ERRO) Impossível escrever pedido de login\n");
144 }
145
146 pthread_t espera_resposta;
147 pthread_create(&espera_resposta, NULL, recebe_resposta, NULL);

```

Figura 5: Função main: conexão com o servidor e recebimento de mensagens.

lido corretamente, faz outro fgets para ler apenas a mensagem (esta foi a alternativa encontrada pois houve vários problemas ao tentar ler a mensagem inteira);

- Os campos do PEDIDO são preenchidos consoante o comando inserido, sendo que a flagVerificado serve como fonte de informação para o manager.

subscribe <tópico>:

- Solicita ao servidor a subscrição de um tópico;
- O servidor retorna mensagens persistentes já existentes no tópico.

unsubscribe <tópico>:

- Cancela a subscrição de um tópico.

Topics:

- Solicita ao servidor a listagem dos tópicos disponíveis.

Help:

- Exibe os comandos disponíveis para o utilizador.

Exit:

- Envia um PEDIDO ao servidor solicitando o encerramento.

```

149 while (1) {
150     PEDIDO p;
151     char comando[20], str[TAM], topico[TAM_USERNAME];
152     int tvida;
153     p.pid = getpid();
154
155     printf("COMANDO 'help' para ver comandos: ");
156
157     if (!fgets(comando, sizeof(comando), stdin)) {
158         continue;
159     }
160
161     if (strcmp(comando, "msg", 3) == 0) {
162         if (scanf(comando, "msg %s %d", topico, &tvda) == 2) {
163             printf("DIGITE A MENSAGEM: ");
164             if (fgets(str, sizeof(str), stdin)) {
165                 str[strlen(str) - 1] = '\0';
166                 strcpy(p.str, str, sizeof(p.str) - 1);
167                 p.str[sizeof(p.str) - 1] = '\0';
168                 strcpy(p.topico, topico);
169                 strcpy(p.nome, login.nome);
170                 if (tvda < 0) {
171                     tvda = 0;
172                 }
173                 p.tempo_vida = tvda;
174                 p.flagVerificado = 1;
175                 escreve_mensagem_manager(p);
176             }
177         } else {
178             printf("Comando inválido. Use correto: msg <TOPICO> <DURACAO>\n");
179         }
180     }
181 }

```

Figura 6: Função main: loop principal.

```

} else if (strcmp(comando, "subscribe", 9) == 0) {
    if (scanf(comando, "subscribe %s", topico) == 1) {
        printf("Subcrevendo o topico %s...\n", topico);
        strcpy(p.str, "Quero subcrever o topico!", 26);
        p.str[26] = '\0';
        strcpy(p.nome, login.nome);
        strcpy(p.topico, topico);
        p.tempo_vida = 0;
        p.flagVerificado = 2;
        escreve_mensagem_manager(p);
    } else {
        printf("Comando inválido. Use correto: subscribe <TOPICO>\n");
    }
} else if (strcmp(comando, "unsubscribe", 11) == 0) {
    if (scanf(comando, "unsubscribe %s", str) == 1) {
        printf("Cancelando a subscricao do topico %s...\n", str);
        strcpy(p.str, "Quero cancelar a subscricao do topico!", 38);
        p.str[38] = '\0';
        strcpy(p.nome, login.nome);
        strcpy(p.topico, topico);
        p.tempo_vida = 0;
        p.flagVerificado = 3;
        escreve_mensagem_manager(p);
    } else {
        printf("Comando inválido. Use correto: unsubscribe <TOPICO>\n");
    }
} else if (strcmp(comando, "topics", 6) == 0) {
    strcpy(p.str, "Quero ver os topicos!", 21);
    p.str[21] = '\0';
    strcpy(p.topico, "");
    strcpy(p.nome, login.nome);
    p.tempo_vida = 0;
    p.flagVerificado = 4;
    escreve_mensagem_manager(p);
}

```

Figura 7: Função main: subscribe/unsubscribe <topics> e topics.

```

} else if (strcmp(comando, "help", 4) == 0) {
    printf("Comandos disponiveis:\n");
    printf(" msg <TOPICO> <DURACAO>\n");
    printf(" subscribe <TOPICO>\n");
    printf(" unsubscribe <TOPICO>\n");
    printf(" topics\n");
    printf(" exit\n");
} else if (strcmp(comando, "exit", 4) == 0) {
    strcpy(p.str, "Vou sair!", 9);
    p.str[9] = '\0';
    strcpy(p.topico, "");
    strcpy(p.nome, login.nome);
    p.tempo_vida = 0;
    p.flagVerificado = -1;
    escreve_mensagem_manager(p);
} else {
    printf("Comando não reconhecido.\n");
}

// Fechar e apagar FIFO do cliente
close(fd);
close(fd_feed);
unlink(fifo_feed);

return 0;
}

```

Figura 8: Função main: help e exit.

Após o fim do ciclo principal os FIFOs são fechados e o do cliente é apagado.

4.2. Função recebe_resposta (thread)

Esta função é executada numa thread separada para receber mensagens do servidor.

```
49 void *recebe_resposta(void *arg){
50     RESPOSTA r;
51     int res;
52
53     while(1){
54         res = read(fd_feed, &r, sizeof(RESPOSTA));
55         if(res == sizeof(RESPOSTA)){
56             if(r.flag == 0){
57                 printf("\nMENSAGEM RECEBIDA: '%s' -> %s (%d)\n", r.str, r.nome, res);
58                 handle_exit(0);
59             }else if(r.flag == 2){
60                 printf("\nMENSAGEM RECEBIDA: TOPICO:%s USER:%s -> '%s' (%d)\n", r.topico, r.nome, r.str, res);
61             }else if(r.flag == 3){
62                 printf("\nMENSAGEM PERSISTENTE RECEBIDA: TOPICO:%s USER:%s TEMPO:%d -> '%s' (%d)\n", r.topico, r.nome, r.tempo, r.str, res);
63             }else{
64                 printf("\nMENSAGEM RECEBIDA: USER:%s -> '%s' (%d)\n", r.nome, r.str, res);
65             }
66         }else{
67             printf("[ERRO] Impossivel ler mensagem!\n");
68         }
69     }
70 }
```

Figura 9: Função recebe_resposta (thread).

Função: lê respostas do FIFO do cliente continuamente.

Comportamento:

- Lê mensagens do tipo RESPOSTA do servidor;
- Baseia-se na flag de RESPOSTA para identificar o tipo de mensagem:
 - Flag 0: notificação de encerramento, procede ao encerramento do feed;
 - Flag 2: mensagens comuns relacionadas a tópicos subscritos;
 - Flag 3: mensagens persistentes;
 - Else: outras mensagens enviadas pelo manager.
- Exibe as mensagens no terminal com informações relevantes;
- Notifica o utilizador em caso de erro ao ler mensagens.

4.3. Função escreve_mensagem_manager

Função utilizada para enviar mensagens ao servidor.

```
9 int escreve_mensagem_manager(PEDIDO p){
10     int res;
11     res = write(fd, &p, sizeof(PEDIDO));
12     if (res == sizeof(PEDIDO)) {
13         if(strlen(p.topico) > 0 && p.tempo_vida > 0){
14             printf("MENSAGEM PERSISTENTE ENVIADA: TOPICO:%s TEMPO:%d -> '%s' (%d)\n", p.topico, p.tempo_vida, p.str, res);
15             return res;
16         }else if(strlen(p.topico) > 0 && p.tempo_vida == 0){
17             printf("MENSAGEM ENVIADA: TOPICO:%s -> '%s' (%d)\n", p.topico, p.str, res);
18             return res;
19         }else if(strlen(p.topico) <= 0){
20             printf("MENSAGEM ENVIADA: '%s' -> MANAGER(%d)\n", p.str, res);
21             return res;
22         }else{
23             printf("MENSAGEM ENVIADA: '%s' ->MANAGER (PID:%d) (%d)\n", p.str, p.pid, res);
24             return res;
25         }
26     }else{
27         printf("[ERRO] Nao foi possivel enviar mensagem!\n");
28         return 0;
29     }
30 }
```

Figura 10: Função escreve_mensagem_manager.

Parâmetro: recebe uma estrutura do tipo PEDIDO com os dados a enviar.

Função:

- Escreve o conteúdo do PEDIDO no FIFO do servidor;
- Exibe uma mensagem no terminal, indicando o sucesso ou falha do envio.

Mensagens Suportadas:

- Mensagens normais e persistentes para tópicos;
- Solicitações de subscrição e cancelamento de subscrição;
- Solicitação de login;
- Pedidos de saída.

4.4. Função `handle_exit`

Manipulador para o sinal SIGINT (Ctrl+C).

Função: garante que o programa encerre de forma ordenada.

Operações:

- Fecha os descritores dos FIFOs abertos;
- Remove FIFO exclusivo do cliente;
- Termina o programa com `exit(0)`.

```
32 void handle_exit(int sig) {
33
34     printf("\nA encerrar o feed...\n");
35
36     if (fd != -1) {
37         close(fd);
38     }
39     if (fd_feed != -1) {
40         close(fd_feed);
41     }
42     if (fifo_created) {
43         unlink(fifo_feed);
44     }
45
46     exit(0);
47 }
```

Figura 11: Função `handle_exit`.

5. MANAGER.C

O **Manager** é o núcleo da plataforma de mensagens baseada em tópicos, responsável por gerir utilizadores, tópicos e mensagens, além de processar comandos administrativos. Ele utiliza **named pipes (FIFOs)** para comunicação com os clientes (**Feed**) e threads para funcionalidades paralelas, como decremento do tempo de vida das mensagens persistentes e execução de comandos administrativos. A leitura de pedidos é feita na função `main`.

As funções mais básicas e genéricas do código são explicadas de maneira mais simples, creio que não necessitam da mesma atenção que outras mais complexas e relevantes no contexto do programa, e claro, respeitando o enunciado.

5.1. Declarações iniciais

Inclusão de bibliotecas:

- Inclui bibliotecas padrão para entrada/saída, manipulação de arquivos (FIFOs), threads e sinais, além de `util.h`, que contém definições e estruturas compartilhadas com os clientes.

```
C manager.c > main(int, char* [])
1  #include "util.h"
2
3  pthread_mutex_t mutex_users = PTHREAD_MUTEX_INITIALIZER;
4  pthread_mutex_t mutex_topics = PTHREAD_MUTEX_INITIALIZER;
5
6  typedef struct {
7      char nome[TAM_USERNAME];
8      pid_t pid;
9      int flag;
10 } USER;
11
12 typedef struct {
13     char str[TAM];
14     char nome_user[TAM_USERNAME];
15     int tempo_vida;
16 } MENSAGEM;
17
18 typedef struct {
19     char nome[TAM_USERNAME];
20     MENSAGEM mensagens[MAX_MENSAGENS];
21     int n_mensagens;
22     int bloqueado;
23     pid_t assinantes[MAX_USERS];
24     int n_assinantes;
25 } TOPICO;
26
27 USER users[MAX_USERS];
28 int n_users = 0;
29 int n_tópicos = 0;
30 TOPICO topicos[MAX_TOPICOS];
```

Figura 12: Manager.c: declarações iniciais.

Inicialização de Mutexes:

- Declara mutexes (mutex_users e mutex_tópicos) para garantir acesso seguro às listas de utilizadores e tópicos durante operações concorrentes realizadas.

Declaração de estruturas:

- **USER:** representa utilizadores conectados;
- **MENSAGEMEMP:** armazena mensagens persistentes;
- **TOPICO:** gere tópicos, mensagens e assinantes.

5.2. Função main

A função principal configura o ambiente do **Manager**, gere o ciclo de vida do servidor, e processa pedidos dos clientes.

Verificação de Servidor Ativo:

- Garante que apenas uma instância do Manager esteja em execução ao verificar se o FIFO_MANAGER já existe.

Criação e Abertura do FIFO Principal:

- Cria o FIFO principal (FIFO_MANAGER) e abre-o para leitura e escrita.

Inicialização de Threads:

- **Comandos manager:** permite ao administrador executar todos os comandos pedidos no enunciado;
- **decrementa_tempo_mensagens:** decrementa o tempo de vida de mensagens persistentes e remove-as quando expiram.

```
732 int main(int argc, char *argv[]) {
733     int fd, fd_feed, res;
734     char fifo_feed[TAM];
735     RESPOSTA rlogin;
736
737     signal(SIGINT, handle_exit);
738
739     if (access(FIFO_MANAGER, F_OK) == 0) {
740         printf("[ERRO] Já existe um servidor!\n");
741         exit(3);
742     }
743
744     if (mkfifo(FIFO_MANAGER, 0666) == -1) {
745         perror("Erro ao criar named pipe\n");
746         exit(1);
747     }
748
749     printf("A aguardar mensagens...\n");
750     fd = open(FIFO_MANAGER, O_RDWR);
751     if (fd == -1) {
752         perror("Erro ao abrir named pipe\n");
753         exit(3);
754     }
755
756     pthread_t thread_comando;
757     pthread_create(&thread_comando, NULL, comandos_manager, NULL);
758
759     pthread_t thread_mensagensp;
760     pthread_create(&thread_mensagensp, NULL, decrementa_tempo_mensagens, NULL);
761
762     carrega_mensagens_persistentes();
```

Figura 13: Manager.c: função main.

Carregamento de Mensagens Persistentes:

- Recupera mensagens persistentes que foram salvas anteriormente no ficheiro MSG_FICH.

5.3. Ciclo principal

Lê estruturas PEDIDO do FIFO principal enviados pelos utilizadores e executa ações baseadas na flagVerificado:

- **Login (flagVerificado == 0):** processamento do login;
- **Envio de Mensagens (flagVerificado == 1):** envia mensagens para tópicos, tratando persistentes e não persistentes;
- **Subscrições de Tópicos (flagVerificado == 2):** adiciona o utilizador como assinantes de um tópico;

- **Cancelamento de Subscrição (flagVerificado == 3):** remove o utilizador do tópico;
- **Listagem de Tópicos (flagVerificado == 4):** envia a lista de tópicos ao cliente;
- **Encerramento de Sessão (flagVerificado == -1):** envia mensagem ao utilizador para que este saiba que tem de sair e limpa-o do sistema.

A imagem mostra como foi feito, promovendo a simplicidade do código.

```

764 while (1) {
765     PEDIDO p;
766     res = read(fd, &p, sizeof(PEDIDO));
767     if (res == sizeof(PEDIDO)) {
768         sprintf(fifo_feed, FIFO_FEED, p.pid);
769         fd_feed = open(fifo_feed, O_WRONLY);
770         if (fd_feed == -1) {
771             perror("Erro ao abrir FIFO do cliente\n");
772             pthread_exit(NULL);
773         }
774
775         if (p.flagVerificado == 0) {
776             if (encontra_nome(p.nome) == -1) {
777                 if (add_user(p.nome, p.pid) {
778                     strcpy(rlogin.str, "USER ADICIONADO");
779                     strcpy(rlogin.nome, "MANAGER");
780                     rlogin.flag = 1;
781                 } else {
782                     strcpy(rlogin.str, "USERS MAX");
783                     strcpy(rlogin.nome, "MANAGER");
784                     rlogin.flag = 0;
785                 }
786             } else {
787                 strcpy(rlogin.str, "USER JA EXISTE");
788                 strcpy(rlogin.nome, "MANAGER");
789                 rlogin.flag = 0;
790             }
791             printf("MENSAGEM ENVIADA: '%s' PARA: %s (PID:%d) (RES:%d)\n", rlogin.str, p.nome, p.pid, res);
792             write(fd_feed, &rlogin, sizeof(RESPOSTA));
793             close(fd_feed);

```

Figura 14: Manager.c: ciclo principal.

Login do utilizador:

- **Ação:** adiciona o utilizador à lista de utilizadores ativos, se possível;
- **Validações:**
 - Verifica se o nome do utilizador já existe;
 - Adiciona o utilizador caso o número máximo permitido não tenha sido atingido.
- **Resposta:** envia uma mensagem ao cliente a informar sucesso ou falha no login.

Envio de Mensagens (flagVerificado == 1):

- **Ação:** envia mensagens para um tópico;
- **Validações:**
 - Verifica se o tópico existe;
 - Verifica se o utilizador está subscrito ao tópico;
 - Confirma se o tópico não está bloqueado.
- **Tratamento de Mensagens:**
 - Mensagens com tempo_vida maior que zero enviadas como persistentes e adicionadas ao tópico;
 - Mensagens com tempo_vida igual a 0 são enviadas diretamente e descartadas.

```

} else if (p.flagVerificado == 1) { //mensagem enviada para tópico
    if (p.tempo_vida > 0) {
        printf("USER %s: TOPICO %s TVIDA %d MSG %s - PID %d\n", p.nome, p.topico, p.tempo_vida, p.str, p.pid);
        if (procura_topico(p.topico) == 1) {
            if (procura_utilizador_topico(p.topico, p.pid) == 0) {
                if (topico_islocked(p.topico) == 1) {
                    envia_mensagem_topico(p.topico, p.str, p.nome, p.tempo_vida);
                    if (add_mensagem_persistente(p.topico, p.nome, p.tempo_vida, p.str) == 1) {
                        printf("Mensagem persistente adicionada com sucesso\n");
                    }
                } else {
                    envia_mensagem("ERRO: O tópico está bloqueado!", p.pid, 0, 0);
                }
            } else {
                envia_mensagem("ERRO: Você não subscreveu o tópico!", p.pid, 0, 0);
            }
        } else {
            envia_mensagem("ERRO: O tópico não existe!", p.pid, 0, 0);
        }
    } else {
        printf("USER %s: TOPICO %s MSG %s - PID %d\n", p.nome, p.topico, p.str, p.pid);
        if (procura_topico(p.topico) == 1) {
            if (procura_utilizador_topico(p.topico, p.pid) == 0) {
                if (topico_islocked(p.topico) == 1) {
                    envia_mensagem_topico(p.topico, p.str, p.nome, 0);
                } else {
                    envia_mensagem("ERRO: O tópico está bloqueado!", p.pid, 0, 0);
                }
            } else {
                envia_mensagem("ERRO: Você não subscreveu o tópico!", p.pid, 0, 0);
            }
        } else {
            envia_mensagem("ERRO: O tópico não existe!", p.pid, 0, 0);
        }
    }
}

```

Figura 15: ciclo principal: envio de mensagens (flagVerificado == 1).

Subscrição de Tópicos (flagVerificado == 2):

- **Ação:** adiciona o utilizador como assinante de um tópico;
- **Validações:**
 - Se o tópico não existe, é criado automaticamente;
 - Se o utilizador já é assinante, nenhuma ação é tomada.
- **Mensagens persistentes:** envia mensagens persistentes do tópico ao utilizador recém-assinado.

Cancelamento de Subscrição (flagVerificado == 3):

- **Ação:** remove o utilizador como assinante de um tópico;
- **Validações:**
 - Verifica se o tópico existe;
 - Remove o utilizador como assinante;
 - Apaga o tópico se não tiver mais assinantes ou mensagens persistentes.

```

else if (p.flagVerificado == 2) { //subscrição de tópicos
    printf("USER: %s MSG: %s TOPICO: %s - PID: %d\n", p.nome, p.str, p.topico, p.pid);
    envia_mensagem("OK", p.pid, 0, 0);

    if (procura_topico(p.topico) == 0) {
        cria_topico(p.pid, p.topico);
        char msg[50];
        sprintf(msg, "Criou e subscreveu o tópico %s", p.topico);
        envia_mensagem(msg, p.pid, 0, 0);
    } else {
        if (procura_utilizador_topico(p.topico, p.pid) >= 0) {
            envia_mensagem("O tópico já foi subscrito", p.pid, 0, 0);
        } else {
            if (utilizador_subscrive_topico(p.topico, p.pid) == 1) {
                sprintf(msg, "Subscriveu o tópico %s", p.topico);
                envia_mensagem(msg, p.pid, 0, 0);
                envia_mensagens_persistentes(p.topico, p.pid);
                envia_mensagem_topico(p.topico, "Subscriveu o tópico", p.nome, 0);
            }
        }
    }
}

else if (p.flagVerificado == 3) { //cancelamento de subscrição
    printf("USER: %s MSG: %s TOPICO: %s - PID: %d\n", p.nome, p.str, p.topico, p.pid);
    envia_mensagem("OK", p.pid, 0, 0);

    if (procura_topico(p.topico) == 0) {
        envia_mensagem("O tópico não existe", p.pid, 0, 0);
    } else {
        if (procura_utilizador_topico(p.topico, p.pid) < 0) {
            envia_mensagem("Você não está subscrito ao tópico", p.pid, 0, 0);
        } else {
            if (utilizador_cancela_sub(p.topico, p.pid) == -1) {
                envia_mensagem("Subscrição cancelada com sucesso", p.pid, 0, 0);
                apaga_topico(p.topico);
            } else {
                envia_mensagem("Subscrição cancelada com sucesso", p.pid, 0, 0);
                envia_mensagem_topico(p.topico, "Cancelou a subscrição", p.nome, 0);
            }
        }
    }
}

```

Figura 16: ciclo principal: subscrição de tópicos (flagVerificado == 2) e cancelamento de subscrição (flagVerificado == 3).

Listagem de tópicos (flagVerificado == 4):

- **Ação:** envia a lista de tópicos ao cliente;
- **Formato:** indica se os tópicos estão bloqueados ou desbloqueados e o número de mensagens persistentes.

```

876         } else if (p.flagVerificado == 4) { //mostra topicos ao utilizador
877             printf("USER: %s MSG: %s TOPICO: %s - PID: %d\n", p.nome, p.str, p.topico, p.pid);
878             envia_mensagem("OK", p.pid, 0, 0);
879             mostra_topicos(p.pid);
880
881         } else if (p.flagVerificado == -1) { //saida do utilizador
882             printf("%s: %s - %d\n", p.nome, p.str, p.pid);
883             envia_mensagem("Voce foi removido pelo manager", p.pid, 1, 0);
884             apaga_utilizador_topicos(p.pid);
885
886         } else {
887             printf("[ERRO] Mensagem desconhecida ou incompleta recebida.\n");
888         }
889     }
890
891     close(fd);
892     handle_exit(0);
893 }
894

```

Figura 17: ciclo principal: listagem de tópicos (flagVerificado == 4) e encerramento de Sessão (flagVerificado == -1).

5.4. Função add_user

Adiciona um novo utilizador à lista de utilizadores ativos.

- Verifica se o número máximo de utilizadores foi atingido;
- Se o utilizador ainda não estiver na lista, adiciona-o num lugar vago e aumenta o contador de utilizadores.

5.5. Função encontra_nome

Procura um utilizador pelo nome.

- Percorre a lista de utilizadores para verificar se existe um utilizador ativo com o nome especificado e retorna o índice do utilizador ou -1 caso não seja encontrado.

5.6. Função mostra_users

Mostra todos os utilizadores ativos.

- Percorre a lista de utilizadores e imprime o nome e o PID dos utilizadores conectados.

5.7. Função cria_topico

Cria um tópico e adiciona o criador como assinante.

- Verifica se o número máximo de tópicos não foi atingido;
- Se o tópico não existir, cria um novo, adicionando-o numa posição vaga encontrada e associa o criador como assinante.

5.8. Função procura_topico

Verifica se um tópico existe.

- Percorre a lista de tópicos e retorna 1 se o tópico for encontrado, ou 0 caso contrário.

5.9. Função procura_utilizador_topicos

Procura um utilizador no tópico especificado.

- Percorre a lista de PID's (assinantes) do tópico e caso encontre o PID especificado retorna a posição em que está, caso contrario retorna -1;
- Se não encontrar o tópico retorna -2.

5.10. Função apaga_utilizador_topicos

Remove um utilizador de todos os tópicos em que está subscrito.

- Para cada tópico, verifica se o utilizador está subscrito e remove-o da lista de assinantes, colocando o PID da posição a 0;
- Decrementa o n_assinantes do tópico;
- Se o tópico não tiver mais assinantes nem mensagens persistentes, é apagado.

5.11. Função `apaga_topico`

Remove um tópico da lista.

- Verifica se o tópico existe e apaga-o da lista, colocando a posição vaga/vazia usando `memset`.

5.12. Função `mostra_topicos`

Mostra todos os tópicos disponíveis.

- Para cada tópico, imprime se está bloqueado ou desbloqueado e o número de mensagens persistentes;
- Envia a lista de tópicos para um cliente, se o PID for especificado, se for `== 0` entende que é para mostrar os tópicos no ecrã.

5.13. Função `lock_topico / unlock_topico`

Bloqueia ou desbloqueia um tópico.

- **`lock_topico`**: muda o campo bloqueado do tópico para 1;
- **`unlock_topico`**: muda o campo bloqueado do tópico para 0.

5.14. Função `topico_islocked`

Verifica se um tópico está bloqueado.

- Retorna 0 se o tópico tiver o campo bloqueado a 1, caso contrário, retorna 1.

5.15. Função `add_mensagem_persistente`

Adiciona uma mensagem persistente a um tópico.

- A função recebe os campos da estrutura `MENSAGEM_P`;
- Encontra o tópico da mensagem;
- As mensagens persistentes são armazenadas no array de mensagens da estrutura do tópico e podem ser recuperadas posteriormente.

5.16. Função `mostra_mensagens_persistentes`

Exibe as mensagens persistentes de um tópico.

- Encontra o tópico especificado e imprime todas as mensagens persistentes associadas e os seus dados (tópico, utilizador, tempo_vida atual e mensagem).

5.17. Função utilizador_cancela_sub

Remove um utilizador da lista de assinantes de um tópico.

- Verifica se o tópico existe;
- Percorre a lista de assinantes do tópico e remove o utilizador com o PID especificado, colocando o PID a 0;
- Decrementa n_assinantes;
- Retorna -1 se o tópico ficou vazio (sem assinantes e sem mensagens) ou retorna 1 se a remoção foi bem-sucedida.

5.18. Função utilizador_subscribe_topico

Adiciona um utilizador como assinante de um tópico.

- Verifica se o tópico existe;
- Percorre a lista de assinantes e adiciona o PID do utilizador na primeira posição livre (== 0);
- Incrementa n_assinantes;
- Retorna 1 se o utilizador foi adicionado com sucesso ou 0 se não foi possível adicioná-lo.

5.19. Função envia_mensagem_todos

Parâmetros:

- **Mensagem:** texto da mensagem a ser enviada a todos os utilizadores;
- **flag:** tipo da mensagem:
 - 1: mensagem de encerramento, preenche a flag de RESPOSTA com 0, para o utilizador saber que tem de sair;
 - 0: mensagem global normal.

Bloqueia a Lista de Utilizadores: garante acesso exclusivo à lista enquanto a mensagem é enviada, através do mutex.

Valida Utilizadores: verifica se há utilizadores conectados; caso contrário, retorna sem enviar.

Itera sobre Utilizadores: percorre a lista de utilizadores e verifica aqueles com nome e PID válidos.

Envia Mensagem a cada utilizador:

- Abre o FIFO exclusivo do utilizador;
- Preenche a estrutura RESPOSTA com a mensagem e o remetente (MANAGER);
- Preenche a flag de RESPOSTA, dependendo da flag enviada;

```
34 int envia_mensagem_todos(const char *mensagem, int flag) {
35     char fifo_feed[TAM];
36     int fd_feed, res;
37     RESPOSTA r;
38
39     pthread_mutex_lock(&mutex_users);
40
41     if (n_users == 0) {
42         pthread_mutex_unlock(&mutex_users);
43         printf("Nao existem utilizadores\n");
44         return 0;
45     }
46
47     for (int i = 0; i < MAX_USERS; i++) {
48         if (strlen(users[i].nome) > 0 && users[i].pid > 0) {
49             sprintf(fifo_feed, FIFO_FEED, users[i].pid);
50             fd_feed = open(fifo_feed, O_WRONLY);
51
52             if (fd_feed == -1) {
53                 perror("Erro ao abrir FIFO");
54                 continue;
55             }
56
57             r.flag = 1;
58             strcpy(r.str, mensagem);
59             strcpy(r.nome, "MANAGER");
60
61             if (flag == 1) {
62                 r.flag = 0;
63             }
64
65             res = write(fd_feed, &r, sizeof(RESPOSTA));
66             close(fd_feed);
67         }
68     }
69
70     pthread_mutex_unlock(&mutex_users);
71     printf("MENSAGEM GLOBAL ENVIADA: '%s' - ALL (%d)\n", r.str, res);
72     return 1;
73 }
```

Figura 18: Função envia_mensagem_todos.

- Escreve no FIFO do utilizador.

Finaliza: fecha o FIFO do utilizador e exhibe no terminal a mensagem enviada.

Desbloqueia a Lista: liberta o mutex.

A função retorna 1 se houver sucesso e 0 se não houver utilizadores conectados.

5.20. Função envia_mensagem

Parâmetros:

- **Mensagem:** mensagem a ser enviada ao utilizador.
- **pidf:** PID do utilizador destinatário;
- **flag1:** indica se a mensagem deve encerrar a sessão do utilizador;
- **flag2:** indica se é necessário notificar todos os utilizadores sobre a remoção do utilizador.

Esta função serve para enviar mensagens para um utilizador ou apagar o mesmo, podendo notificar ou não os outros utilizadores através da função `envia_mensagem_todos`. Foi feito desta maneira para não haver necessidade de repetir linhas de código desnecessariamente, sendo que a maneira que o manager notifica os utilizadores que tem de sair é também uma mensagem PEDIDO.

A função `envia_mensagem` envia uma mensagem específica para um utilizador identificado pelo seu PID.

Bloqueia a Lista de Utilizadores: garante que outros processos não alterem a lista enquanto a mensagem está a ser enviada.

Procura o Utilizador: verifica se o utilizador com o PID fornecido está conectado e ativo.

Prepara e Envia a Mensagem:

- Abre o FIFO exclusivo do utilizador;
- Preenche a estrutura `RESPOSTA` com a mensagem, remetente (`MANAGER`), e a flag correspondente;
- Escreve a mensagem no FIFO.

Remove o Utilizador (Se `flag1 == 1`):

- Preenche a flag de `RESPOSTA` com 0, para o feed saber que tem de sair;
- Reseta os dados do utilizador na lista;
- Reduz o contador de utilizadores ativos.

Notifica outros utilizadores (Se `flag2 == 1`): usa `envia_mensagem_todos` para informar a remoção do utilizador aos outros.

Finaliza: fecha o FIFO do utilizador e desbloqueia a lista.

Return:

- Retorna 1 se a mensagem foi enviada com sucesso ou o utilizador foi encontrado;
- Retorna 0 se o utilizador não foi encontrado ou houve erro.

5.21. Função salva_mensagens_persistentes

Esta função garante que as mensagens persistentes sejam salvas de forma segura e possam ser recuperadas posteriormente.

Abre o arquivo MSG_FICH para escrita.

Bloqueia o mutex mutex_topicos para garantir acesso seguro à lista de tópicos.

Itera sobre os tópicos e, para cada tópico com mensagens persistentes, grava no arquivo o nome do tópico, o nome do usuário, o tempo de vida da mensagem e o conteúdo da mensagem.

```
177 void salva_mensagens_persistentes() {
178     FILE *f = fopen("MSG_FICH", "w");
179     if (!f) {
180         perror("Erro ao abrir MSG_FICH");
181         return;
182     }
183
184     pthread_mutex_lock(&mutex_topicos);
185
186     for (int i = 0; i < MAX_TOPICOS; i++) {
187         if (strlen(topicos[i].nome) > 0) {
188             for (int j = 0; j < topicos[i].n_mensagens; j++) {
189                 fprintf(f, "%s %s %d %s\n", topicos[i].nome,
190                     topicos[i].mensagens[j].nome_user,
191                     topicos[i].mensagens[j].tempo_vida,
192                     topicos[i].mensagens[j].str);
193             }
194         }
195     }
196
197     pthread_mutex_unlock(&mutex_topicos);
198     fclose(f);
199 }
```

Desbloqueia o mutex após terminar de gravar as mensagens. **Figura 19:** Função salva_mensagens_persistentes.

Fecha o arquivo.

5.22. Função carrega_mensagens_persistentes

A função carrega_mensagens_persistentes é responsável por carregar as mensagens persistentes do arquivo MSG_FICH e restaurá-las nos tópicos correspondentes, criando-os.

Abre o Arquivo MSG_FICH:

- Abre o arquivo em modo de leitura ("r");
- Se o arquivo não existir, exibe uma mensagem informando que não há mensagens a carregar.

Lê as Mensagens do Arquivo:

- Cada linha do arquivo contém:
 - Nome do tópico;
 - Autor da mensagem;
 - Tempo de vida da mensagem;
 - Texto da mensagem.

```
199 void carrega_mensagens_persistentes() {
200     FILE *f = fopen("MSG_FICH", "r");
201     if (!f) {
202         printf("Arquivo MSG_FICH nao encontrado\n");
203         return;
204     }
205
206     char topico[TAM_USERNAME], mensagem[TAM], autor[TAM_USERNAME];
207     int tempo_vida;
208     while (fscanf(f, "%s %s %d %s\n", topico, autor, &tempo_vida, mensagem) == 4) {
209         for (int i = 0; i < MAX_TOPICOS; i++) {
210             if (strcmp(topicos[i].nome, topico) == 0 || strlen(topicos[i].nome) == 0) {
211                 if (strlen(topicos[i].nome) == 0) {
212                     strcpy(topicos[i].nome, topico);
213                     n_topicos++;
214                 }
215                 strcpy(topicos[i].mensagens[topicos[i].n_mensagens].str, mensagem);
216                 strcpy(topicos[i].mensagens[topicos[i].n_mensagens].nome_user, autor);
217                 topicos[i].mensagens[topicos[i].n_mensagens].tempo_vida = tempo_vida;
218                 topicos[i].n_mensagens++;
219                 break;
220             }
221         }
222     }
223     fclose(f);
224 }
```

Figura 20: Função carrega_mensagens_persistentes.

Restaura as Mensagens:

- Para cada mensagem lida:
 - Procura o tópico correspondente na lista;
 - Se o tópico não existir, cria-o automaticamente;

- Adiciona a mensagem à lista de mensagens persistentes do tópico.

Fecha o arquivo: após carregar todas as mensagens, o arquivo é fechado.

5.23. Função `handle_exit`

Lida com o encerramento do servidor.

- Salva mensagens persistentes no arquivo `MSG_FICH`;
- Envia uma mensagem de encerramento a todos os utilizadores, provocando a sua saída;
- Libera todos os recursos, remove os tópicos e utilizadores, e encerra o servidor de forma ordenada.

5.24. Função `envia_mensagem_topico`

A função `envia_mensagem_topico` envia uma mensagem para todos os utilizadores que estão subscritos a um tópico específico.

Bloqueia o Mutex: usa `mutex_topicos` para proteger a lista de tópicos contra alterações concorrentes.

Encontra o tópico: verifica se o tópico especificado existe na lista.

Itera sobre os Assinantes do Tópico: para cada assinante, abre o FIFO exclusivo do utilizador e envia a mensagem.

Preenche a Estrutura de Resposta:

- Caso a mensagem seja não persistente, coloca a flag de `RESPOSTA` a 2 (para que o feed a processe como não persistente);
- Caso a mensagem seja persistente, coloca a flag de `RESPOSTA` a 3 e preenche o tempo de vida respetivamente.

Envia a mensagem: escreve no FIFO exclusivo do assinante a estrutura `RESPOSTA`, contendo a mensagem e informações adicionais.

Desbloqueia o Mutex: liberta o acesso à lista de tópicos após o envio.

Return: retorna 1 se a mensagem foi enviada ou 0 se o tópico não foi encontrado ou ocorrer um erro.

```

201 void handle_exit(int sig) {
202     printf("A encerrar o servidor...\n");
203
204     salva_mensagens_persistentes();
205
206     envia_mensagem_todos("O servidor foi encerrado!!", 1);
207
208     for (int i = 0; i < MAX_USERS; i++) {
209         if (strlen(users[i].nome) > 0) {
210             memset(&users[i], 0, sizeof(USER));
211         }
212     }
213
214     for (int i = 0; i < MAX_TOPICOS; i++) {
215         if (strlen(topicos[i].nome) > 0) {
216             memset(&topicos[i], 0, sizeof(TOPICO));
217         }
218     }
219
220     unlink(FIFO_MANAGER);
221     pthread_mutex_destroy(&mutex_users);
222     pthread_mutex_destroy(&mutex_topicos);
223     exit(0);
224 }

```

Figura 21: Função `handle_exit`.

```

455 int envia_mensagem_topico(char t[TAM_USUENAME], const char *msg, const char *user, int tvida){
456     char fifo_feed[TAM];
457     int fd_feed, res, encontra = 0;
458     RESPOSTA r;
459
460     pthread_mutex_lock(&mutex_topicos);
461
462     for (int i = 0; i < MAX_TOPICOS; i++){
463         if (strcmp(t, topicos[i].nome) == 0){
464             for(int j = 0; j < MAX_USERS; j++){
465                 if (topicos[i].assinantes[j] > 0){
466                     sprintf(fifo_feed, FIFO_FEED, topicos[i].assinantes[j]);
467                     fd_feed = open(fifo_feed, O_WRONLY);
468
469                     if (fd_feed == -1) {
470                         pthread_mutex_unlock(&mutex_topicos);
471                         perror("Erro ao abrir FIFO\n");
472                         return 0;
473                     }
474                     strcpy(r.str, msg);
475                     strcpy(r.nome, user);
476                     strcpy(r.topico, t);
477                     r.flag = 2;
478
479                     if (tvda > 0){
480                         r.flag = 3;
481                         r.tempo = tvda;
482                     }
483
484                     res = write(fd_feed, &r, sizeof(RESPOSTA));
485                     printf("MENSAGEM ENVIADA: '%s' PARA: %s (PID:%d) (RES:%d)\n",
486                           r.str, t, topicos[i].assinantes[j], res);
487                     close(fd_feed);
488                 }
489             }
490         }
491     }
492     pthread_mutex_unlock(&mutex_topicos);
493     return 1;
494 }
495
496 }

```

Figura 21: Função `envia_mensagem_topico`.

5.25. Função envia_mensagens_persistentes

Pârametros:

- **t**: nome do tópico cujas mensagens persistentes serão enviadas;
- **pidf**: PID do utilizador que receberá as mensagens.

Bloqueia o Mutex: usa mutex_topicos para proteger a lista de tópicos.

Encontra o tópico: verifica se o tópico especificado existe e se não existir, retorna -1.

Verifica mensagens persistentes: caso o tópico não possua mensagens persistentes, retorna 0.

Abre o FIFO do Utilizador: usa o PID do utilizador para abrir o seu FIFO exclusivo para escrita.

Itera sobre as Mensagens Persistentes:

- Para cada mensagem persistente do tópico:
 - Preenche a estrutura RESPOSTA com os dados da mensagem;
 - Escreve no FIFO do utilizador.

Desbloqueia o Mutex e Fecha o FIFO.

Return:

- Retorna 1 se as mensagens foram enviadas com sucesso;
- Retorna -1 se o tópico não existir e 0 se não houver mensagens a enviar.

```

557 int envia_mensagens_persistentes(char t[TAM_USERNAME], pid_t pidf){
558     char fifo_feed[TAM];
559     int fd_feed, res, encontra = 0;
560     RESPOSTA r;
561
562     pthread_mutex_lock(&mutex_topicos);
563
564     for (int i = 0; i < MAX_TOPICOS; i++){
565         if(strcmp(t, topicos[i].nome) == 0){
566             if(topicos[i].n_mensagens == 0){
567                 pthread_mutex_unlock(&mutex_topicos);
568                 return 0;
569             }else{
570                 sprintf(fifo_feed, FIFO_FEED, pidf);
571                 fd_feed = open(fifo_feed, O_WRONLY);
572
573                 if (fd_feed == -1) {
574                     pthread_mutex_unlock(&mutex_topicos);
575                     perror("Erro ao abrir FIFO\n");
576                     return 0;
577                 }
578
579                 for(int j = 0; j < topicos[i].n_mensagens; j++){
580                     strcpy(r.str, topicos[i].mensagens[j].str);
581                     strcpy(r.nome, topicos[i].mensagens[j].nome_user);
582                     strcpy(r.topico, topicos[i].nome);
583                     r.tempo = topicos[i].mensagens[j].tempo_vida;
584                     r.flag = 3;
585
586                     res = write(fd_feed, &r, sizeof(RESPOSTA));
587
588                 }
589                 pthread_mutex_unlock(&mutex_topicos);
590                 close(fd_feed);
591                 return 1;
592             }
593         }
594     }
595     pthread_mutex_unlock(&mutex_topicos);
596     return -1;
597 }

```

Figura 22: Função envia_mensagens_persistentes.

5.26. Threads

5.26.1. decrementa_tempo_mensagens

Gere o tempo de vida das mensagens persistentes e remove aquelas que expirarem. Caso o tópico não tenha mensagens persistentes ou assinantes chama a função apaga_topico, garantindo que as mensagens persistentes funcionem como esperado.

Decrementa o tempo de vida das mensagens:

- Para cada tópico, verifica todas as mensagens persistentes;
- Reduz o tempo de vida das mensagens a cada segundo.

```

void *decrementa_tempo_mensagens(void *arg) {
    while (1) {
        int flag = 0;
        char t[TAM_USERNAME];
        pthread_mutex_lock(&mutex_topicos);

        for (int i = 0; i < MAX_TOPICOS; i++) {
            if (strlen(topicos[i].nome) > 0) { // Verifica tópico
                for (int j = 0; j < topicos[i].n_mensagens; j++) {
                    if (topicos[i].mensagens[j].tempo_vida > 0) {
                        topicos[i].mensagens[j].tempo_vida--;

                        // Remove se o tempo de vida chegar a 0
                        if (topicos[i].mensagens[j].tempo_vida == 0) {
                            printf("Mensagem expirada no Tópico %s: %s\n", topicos[i].nome,
                                topicos[i].mensagens[j].str);

                            // Remover mensagens deslocando
                            for (int k = j; k < topicos[i].n_mensagens - 1; k++) {
                                topicos[i].mensagens[k] = topicos[i].mensagens[k + 1];
                            }
                            topicos[i].n_mensagens--;
                        }
                        if (topicos[i].n_mensagens == 0 && topicos[i].n_assinantes == 0) {
                            strcpy(t, topicos[i].nome);
                            flag = 1;
                        }
                        j--; // Ajusta i índice para continuar a verificar mensagens
                    }
                }
            }
        }

        pthread_mutex_unlock(&mutex_topicos);

        if(flag == 1){
            if(apaga_topico(t) == 1){
                printf("Tópico %s apagado!\n", t);
            }
        }

        // Espera 1 seg
        sleep(1);
    }

    return;
}

```

Figura 23: Threads: decrementa_tempo_mensagens.

Remove mensagens expiradas: quando o tempo de vida de uma mensagem atinge zero, ela é apagada.

Apaga tópicos vazios: se um tópico não tiver mais mensagens nem assinantes, ele é removido automaticamente.

Espera 1 segundo por ciclo: usa `sleep(1)` para evitar consumo excessivo de CPU.

5.26.2. comandos_manager

Permite ao administrador interagir com o servidor e ter controlo total para gerir os utilizadores, tópicos e mensagens. Executa as respetivas ações com base no comando fornecido.

Lê os comandos escritos no terminal.

Processa os comandos:

- **romver<PID>:** remove um utilizador;
- **users:** mostra os utilizadores conectados;
- **topics:** mostra os tópicos disponíveis;
- **show <TOPICO>:** mostra mensagens persistentes de um tópico;
- **lock <TOPICO> e unlock <TOPICO>:** bloqueiam ou desbloqueiam um tópico;
- **close:** encerra o servidor através de `handle_exit`.

```
void *comandos_manager(void *arg) {
    char comando[TAM_USERNAME];
    char str[TAM], topico[TAM_USERNAME];
    pid_t pid;

    while (1) {
        printf("Comando: ");
        if (!fgets(comando, sizeof(comando), stdin)) {
            continue;
        }

        if (strcmp(comando, "remove", 7) == 0) {
            if (sscanf(comando, "remove %d", &pid) == 1) {
                envia_mensagem("Voce foi expulso pelo manager", pid, 1, 1);
                apaga_utilizador_topicos(pid);
            } else {
                printf("Comando invalido. Uso correto: remove <PID>\n");
            }
        } else if (strcmp(comando, "users", 5) == 0) {
            mostra_users();
        } else if (strcmp(comando, "topics", 6) == 0) {
            printf("Listando topicos...\n");
            mostra_topicos(0);
        } else if (strcmp(comando, "show", 4) == 0) {
            if (sscanf(comando, "show %s", topico) == 1) {
                printf("Mostrando o topico %s\n", topico);
                int resultado = mostra_mensagens_persistentes(topico);
                if (resultado == 0) {
                    printf("Nao existem mensagens persistentes no topico %s\n", topico);
                } else if (resultado == -1) {
                    printf("Impossivel mostrar mensagens persistentes\n");
                }
            } else {
                printf("Comando invalido. Uso correto: show <TOPICO>\n");
            }
        } else if (strcmp(comando, "lock", 4) == 0) {
            if (sscanf(comando, "lock %s", topico) == 1) {
                printf("Bloqueando o topico %s\n", topico);
                lock_topico(topico);
                envia_mensagem_topico(topico, "O topico foi bloqueado!", "MANAGER", 0);
            } else {
                printf("Comando invalido. Uso correto: lock <TOPICO>\n");
            }
        } else if (strcmp(comando, "unlock", 6) == 0) {
            if (sscanf(comando, "unlock %s", topico) == 1) {
                printf("Desbloqueando o topico %s\n", topico);
                envia_mensagem_topico(topico, "O topico foi desbloqueado!", "MANAGER", 0);
                unlock_topico(topico);
            } else {
                printf("Comando invalido. Uso correto: unlock <TOPICO>\n");
            }
        } else if (strcmp(comando, "close", 5) == 0) {
            handle_exit(0);
        } else {
            printf("Comando nao reconhecido.\n");
        }
    }

    return NULL;
}
```

Figura 24: Threads: comandos_manager.

6. CONCLUSÃO

O desenvolvimento deste sistema permitiu aplicar conceitos fundamentais de comunicação entre processos e sincronização em ambientes UNIX/Linux. A implementação do Manager e do Feed garantiu a troca eficiente de mensagens, a gestão de utilizadores, tópicos e mensagens persistentes, atendendo aos requisitos do enunciado.

Este trabalho ofereceu uma experiência prática relevante, poupando também muito tempo de estudo para o exame.